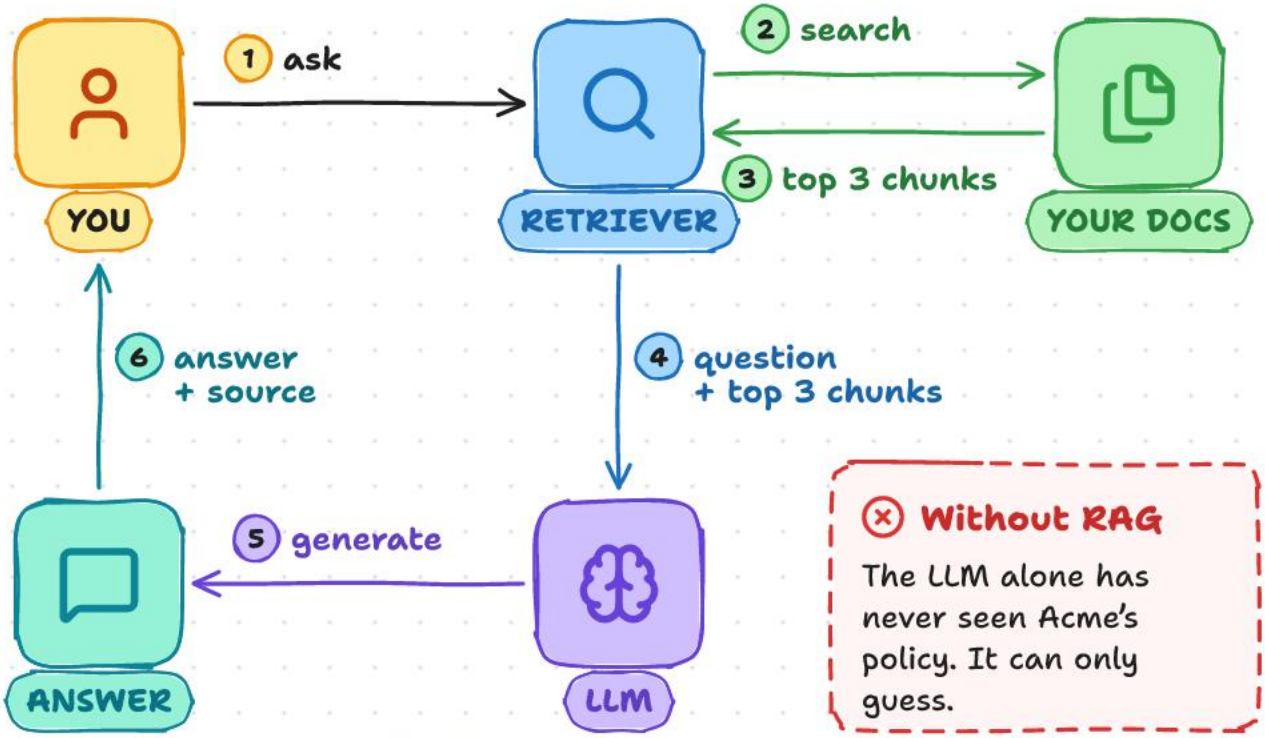
How RAG answers a question

Retrieval-Augmented Generation: look it up first, then answer.



Without RAG

The LLM alone has never seen Acme's policy. It can only guess.

(Day 01 uses an API-free stand-in)

REAL RUN · Day 01 notebook

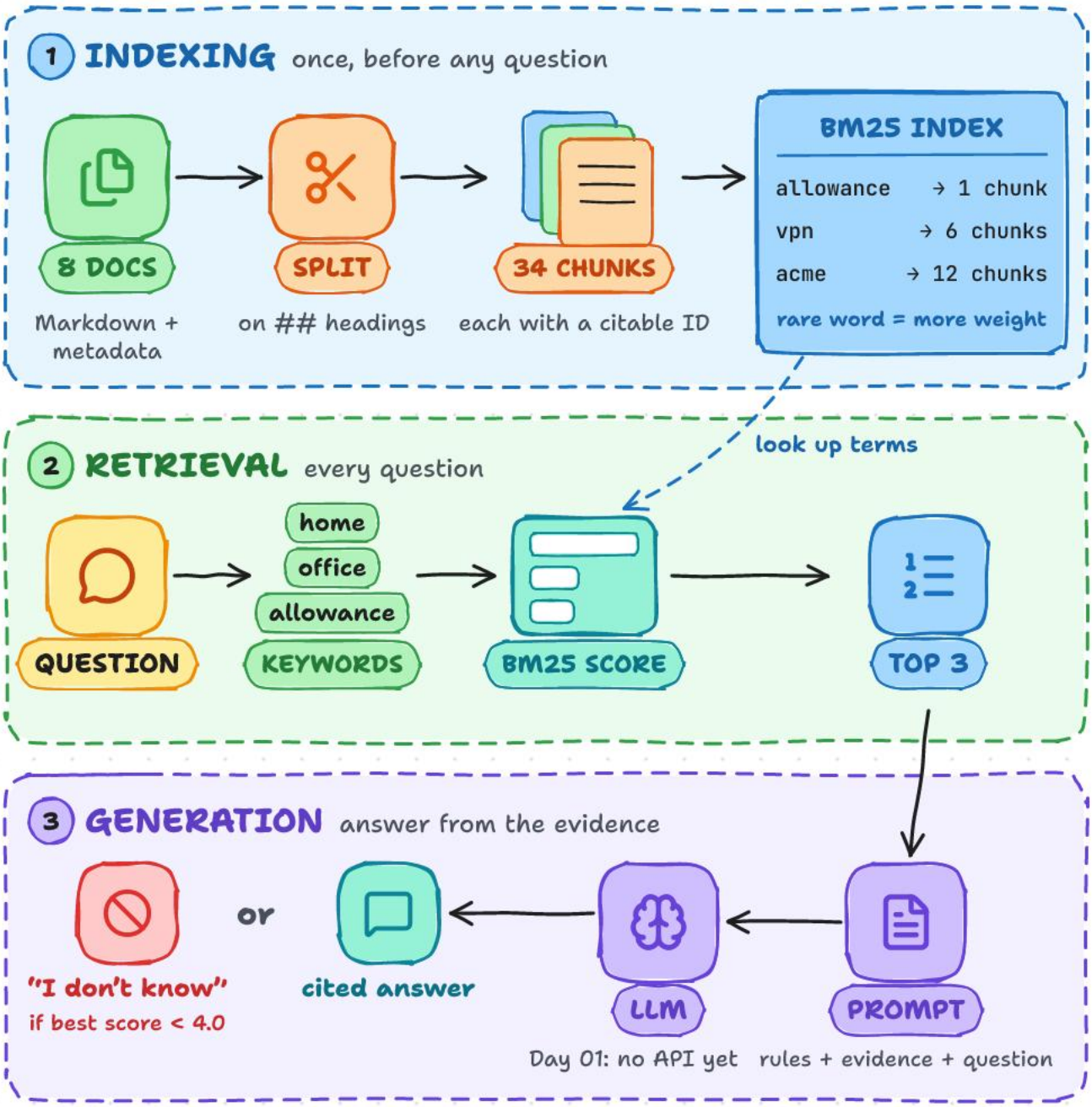
Q: How much is the home office allowance for remote employees?

A: "... a one-time home office allowance of USD 500 ..."

source `remote-work-policy#equipment-and-home-office`

Index once, answer many times

The three stages of RAG, as built in the Day 01 notebook.



What happens to one question

A real run from the Day 01 notebook. No API, no embeddings.

How much is the home office allowance for remote employees?

1 split into keywords

~~much~~ | home | office | allowance | remote | employees

in no document: adds 0

rare (1 of 34 chunks): counts most

2 score every chunk with BM25

#1 remote-work-policy#equipment-and-home-office — 13.43

#2 remote-work-policy#security-while-remote — 4.92

#3 remote-work-policy#core-hours — 4.29

4.0 = MIN_SCORE

If even #1 scored below the red line: "I don't know".

3 answer from the top chunk, with its source

"Employees approved for remote work receive a one-time home office allowance of USD 500 for a desk, chair or monitor."

source: remote-work-policy#equipment-and-home-office

3 ways it breaks, on purpose

Real runs from the notebook. Each one sets up a later day.

1 Wrong words: evidence never found



→ Later: embeddings, hybrid search, query rewriting (Days 04, 08, 10)

2 Relevant is not the same as answering



→ Later: a reader that checks the evidence, plus an eval set (Days 06, 07)

3 "I don't know" is a feature



→ Keep this behaviour as the system grows.

Next → Day 02: ingestion, parsing and metadata